OWASP Top 10 Compliance Checklist

Guideline reference: §8.1 (OWASP Top 10 compliance), §8.2 (code quality gates). Evidence for the final report (§10.1, item 8).

- **Date of review:** 2026-10-27
- **Scope:** `edura-backend` (11 services), `edura-frontend`, `edura-infra` (gateway, Kubernetes, pipelines), `edura-tests`.
- **Method:** static review of the source, configuration and CI workflows, plus a count of the tests that assert the behaviour. The staging environment was **not** probed (no live requests were sent), so runtime behaviour such as error bodies and response headers is marked where it was not observed.
- **Status key: Met** = implemented and evidenced. **Partly met** = implemented with a stated gap. **Not met** = required by the guideline but not implemented. A line number is given where a file is cited.

Summary

ID	Risk	Status
A01	Broken Access Control	Met
A02	Cryptographic Failures	Partly met (hardcoded fallback signing key)
A03	Injection	Met (no dedicated injection test)
A04	Insecure Design	Not met (no threat model in the SDS)
A05	Security Misconfiguration	Partly met
A06	Vulnerable and Outdated Components	Partly met
A07	Identification and Authentication Failures	Partly met
A08	Software and Data Integrity Failures	Partly met
A09	Security Logging and Monitoring Failures	Not met
A10	Server-Side Request Forgery	Met

A01: Broken Access Control

Required: role-based access checks on every endpoint; tests verify unauthorised access returns 403.

Evidence - All 174 endpoints were enumerated from the FastAPI routes (`testing/endpoint-register.md`). **132 have an authentication or role dependency** (`require_role` / token verification). The other **42 were each reviewed** and are public by design: - 11 `/health` checks, one per service. - 11 sign-in related endpoints in `auth_service`: register, verify-email, resend-verification, login, refresh, google, forgot-password, reset-password, logout, otp/request, otp/verify. - 15 read-only catalogue endpoints: course list, course

detail, modules, lessons and materials listings (`course_service`); streams, grade subjects, subjects (`user_service`); assessment list, rankings (exams and leaderboard) and badges (`assessment_service`); class packages (`enrollment_service`); bank accounts (`payment_service`); platform statistics (`analytics_service`). - `POST /webhook` (payment): authenticated by the PayHere HMAC signature, not a user token. - `GET /internal/admin-stats` (payment): service-to-service only; the gateway blocks `^/api/[^/]+/internal` (`edura-infra/nginx/nginx.conf`, `k8s/base/nginx.conf`). - `POST /feedback/submit`, `POST /chat/send`, `GET /chat/history` (`admin_service`): allow guests by design; a signed-in user is linked to the record, a guest uses a session id. - The service tests contain **72 assertions of HTTP 403 and 51 of HTTP 401** (for example `test_admin_endpoint_forbids_other_roles`, `test_list_feedback_requires_admin`).

Limitations - Whether the public list endpoints hide unpublished content was not reviewed here. - The sweep was a static review; no automated test enumerates every write route and fails on an unprotected one.

A02: Cryptographic Failures

Required: passwords hashed with bcrypt (cost ≥ 12); tokens stored as hashes; TLS enforced.

Evidence - Passwords are hashed with `bcrypt.hashpw(..., bcrypt.gensalt())` (`auth_service/router.py`, registration and password reset). The library default work factor is **12**, which meets the minimum. - Refresh tokens are stored as **SHA-256 hashes** (`RefreshToken.token_hash`, `auth_service/models.py`; `services.py`). - Public traffic is served over **HTTPS**: Caddy terminates TLS with a Let's Encrypt certificate (`edura-infra/k8s/base/50-edge.yaml`). See ADR-13. - Real secrets are not committed: `.env` files are git-ignored in every repository, and each repository commits an `.env.example` with placeholders. Deployed environments receive `SECRET_KEY` and the payment credentials from GitHub Secrets. The test suites no longer carry secret defaults (DDP-#54). - The payment webhook signature (PayHere HMAC-MD5) is checked before an event is accepted (`payment_service/utils.py`).

Gaps - **Hardcoded fallback signing key.** `shared/auth.py` (used by every service) and `auth_service/config.py` read `SECRET_KEY` from the environment but fall back to a fixed development value if it is unset. If a deployment ever started without the variable, tokens could be forged with a publicly known key. This is a configuration risk, not an active vulnerability, as long as every environment sets its own key. Someone with access should confirm that staging's `SECRET_KEY` differs from the default. - The bcrypt cost depends on the library default and is not set explicitly in code. - Access tokens are signed with HS256 and a shared `SECRET_KEY`. - The webhook signature is compared with `==`, not a constant-time comparison.

A03: Injection

Required: parameterised queries or ORM throughout; input validated and sanitised.

Evidence - Database access goes through the SQLAlchemy ORM. The only raw SQL is `text(stmt)` in service start-up code (`auth_service`, `assessment_service`, `payment_service`, `course_service`, `notification_service`), and every statement is a **literal string constant** (`_COLUMN_MIGRATIONS` and the enum statements); no user input reaches them. - Request bodies and parameters are validated by Pydantic schemas and FastAPI parameter types.

Limitations - No test sends injection payloads (for example `' OR 1=1--`) to list or search endpoints. Adding one would turn this from reviewed to tested.

A04: Insecure Design

Required: threat model documented in the SDS security section; design reviewed against it.

Evidence / gap - The SDS has a project risk register but **no threat model** (for example STRIDE or a data-flow analysis). - Design decisions that reduce risk are recorded in ADRs (ADR-12 event-driven communication, ADR-13 deployment, ADR-14 accepted vulnerabilities).

Action: add a short threat model to the SDS security section.

A05: Security Misconfiguration

Required: default credentials removed; error messages do not leak stack traces in production.

Evidence - CORS at the gateway allows only the public frontend origin (`Access-Control-Allow-Origin` set to the public URL in `k8s/base/nginx.conf`), with credentials. - Rate limiting: 100 requests per minute per IP, burst 20, status 429 (`nginx.conf`). - No debug mode or custom exception handler that returns tracebacks was found in the service entry points. FastAPI returns a generic body for unhandled errors by default; this was **not observed** on staging. - Internal-only endpoints are blocked at the gateway (see A01).

Gaps - **RabbitMQ still uses `guest/guest`.** It is reachable only inside the cluster, but the guideline asks for default credentials to be removed (also listed in `edura-infra/docs/AZURE-DEPLOYMENT.md`). - **No security response headers** (HSTS, `X-Content-Type-Options`, a basic CSP) were found in the gateway or Caddy configuration.

A06: Vulnerable and Outdated Components

Required: dependency scan in the CI pipeline; no high or critical vulnerabilities.

Evidence - Every code repository scans dependencies in CI: `pip-audit` for the backend (all 11 services), `edura-infra` and `edura-tests`; `audit-ci` (npm) for the frontend. Each fails the build on a new advisory. - Accepted exceptions are written down, not hidden: frontend advisories in `audit-ci.jsonc`; infra and tests in `security/accepted-vulnerabilities.txt` (currently empty; DDP-#56 and DDP-#57, pending merge when this checklist was written); procedure in each repository's `docs/SECURITY-SCAN.md`; rationale in ADR-14.

Gap - The frontend allow-list (`audit-ci.jsonc`) holds **22 accepted advisories** (ADR-14: `next`, `dompurify`, `follow-redirects`, `js-yaml`, `postcss`, `sharp` and others). The scan fails on high or critical advisories, so these are documented exceptions to "no high or critical vulnerabilities".

A07: Identification and Authentication Failures

Required: account lockout after N failed attempts; secure session management; MFA optional.

Evidence - Refresh tokens are **rotated and revoked** on use (`rotate_refresh_token`, `is_revoked`, `auth_service/services.py`). - Access tokens are short-lived (`ACCESS_TOKEN_EXPIRE_MINUTES`); refresh tokens expire after `REFRESH_TOKEN_EXPIRE_DAYS`. - One-time codes expire after **5 minutes** (`OTP_TTL_SECONDS = 300`, stored in Redis). - The gateway rate-limits requests per IP (see A05).

Gap - **No account lockout.** The login endpoint returns `INVALID_CREDENTIALS` on a wrong password but does not count failures; only the gateway rate limit slows repeated attempts.

A08: Software and Data Integrity Failures

Required: package lock files committed; pipeline verifies integrity.

Evidence - The frontend commits `package-lock.json`, and CI installs with `npm ci`, which fails on a lock mismatch. - Container images are built in CI and tagged with the git commit SHA (`edura-infra` deploy pipeline), so a deployed image maps to one commit. - The payment webhook verifies the PayHere HMAC signature before accepting the event.

Gap - The backend has **no Python lock file**; the `requirements.txt` files mix pinned (`==`) and unpinned entries, so builds are not fully reproducible.

A09: Security Logging and Monitoring Failures

Required: all auth events, errors and data access logged; logs do not contain PII or secrets.

Evidence - Errors and service start-up events are logged through Python `logging` in the services.

Gaps - **Authentication events are not logged.** `auth_service` logs only start-up messages; login success and failure, token refresh and password changes are not recorded. - **Logs contain PII.** `notification_service` logs recipient email addresses (`email_service.py`: "Sending email … to {to_email}"; `consumer.py`: "… for {email}"). - There is no central log collection or alerting beyond Kubernetes pod logs.

A10: Server-Side Request Forgery

Required: external URL inputs validated; allow-list of permitted domains.

Evidence - The services make only two kinds of outbound HTTP calls, both to **fixed addresses** that users cannot influence: Google's userinfo endpoint (`GoogleAuthService.USERINFO_URL`) and the Resend email API (`RESEND_API_URL`, from configuration). - No endpoint accepts a URL from a user and fetches it; video lessons are stored as a YouTube video **id**, not a URL.

Consolidated gaps and recommended actions

#	Gap	OWASP	Suggested action
1	No account lockout	A07	Record as a limitation (application code is frozen); add failed-attempt counting in a later release
2	Authentication events not logged; recipient emails in logs	A09	Record as a limitation; later: log auth events, mask emails
3	No threat model in the SDS	A04	Add a short threat model to the SDS security section
4	No security response headers	A05	Add headers in the Caddy/Nginx configuration (no application change)
5	RabbitMQ `guest/guest`	A05	Change in the Kubernetes secret and `.env` (configuration change)

#	Gap	OWASP	Suggested action
6	22 accepted high-severity frontend advisories	A06	Keep ADR-14 up to date; remove entries as upstream fixes land
7	No Python lock file	A08	Add pinned requirements or a lock file in a later release
8	No injection-payload test	A03	Add one negative test per search or list endpoint
9	bcrypt cost relies on the library default	A02	Set `rounds=12` explicitly in a later release
10	Hardcoded fallback `SECRET_KEY` in `shared/auth.py` and `auth_service/config.py`	A02	Verify every environment sets its own key (configuration check); remove the default in a later release so a missing key fails at start-up

Items 1, 2, 7, 9 and 10 need application code changes, which the team has deferred to avoid destabilising the running system; they should be listed under "Limitations & future work" in the report and the demonstration (§9.2).